

© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2022

V. Sarcar, *Java Design Patterns*

https://doi.org/10.1007/978-1-4842-7971-7_29

29. Anti-Patterns

Vaskaran Sarcar¹

(1) Garia, Kolkata, India

The discussion of design patterns is not complete without discussing anti-patterns. This chapter covers a brief overview of anti-patterns. Let's start.

Overview

In real-world application development, some approaches are very attractive at the beginning, but in the long run, they create problems. Consider the case when you try to get a quick fix to meet a delivery deadline. If you are not aware of the potential pitfalls, you may pay a big penalty.

Anti-patterns alert you to common mistakes by describing how attractive approaches can make your life difficult in the future. At the same time, they suggest alternate solutions that may seem tough or ugly at the beginning but ultimately help you build a better solution. So, you must take precautionary measures to avoid those mistakes. The proverb “prevention is better than cure” very much suits this context.

Points to Remember

In short, anti-patterns identify problems with established practices and can map general situations to a specific class of highly productive solutions. They can also provide you with better plans to reverse bad practices and make healthy solutions. In addition, remember that the concept of anti-patterns is not limited to object-oriented programming.

Brief History of Anti-Patterns

The original idea of design patterns came from building architect Christopher Alexander, a professor at the University of California, Berkeley. He shared his ideas for the construction of buildings within well-planned towns. Gradually these concepts entered into software development, and they started gaining popularity through leading-edge software developers like Ward Cunningham and Kent Beck. In 1994, the idea of design patterns entered into the mainstream of object-oriented software development through an industry conference called Pattern Languages of Program Design (PLoP) on design patterns. It was hosted by the Hillside Group, and Jim Coplien's paper "A Development Process Generative Pattern Language" is a famous one in this context. With the launch of the classic textbook *Design Patterns: Elements of Reusable Object-Oriented Software* by the GoF, the idea of design patterns became extremely popular.

Undoubtedly, the idea of design patterns helped (and are still helping) programmers develop high-quality software. But in some cases, people started noticing the negative impacts, also. Here is an example: many developers wanted to show their expertise without a true evaluation of the consequences of these patterns in their specific domains. As an obvious side-effect, patterns were implemented in the wrong context, produced low-quality software, and ultimately caused big penalties to them or their organizations.

So, the software industry needed to focus on the negative consequences of similar mistakes and eventually the idea of anti-patterns evolved. Many experts contributed to this field, but the first well-formed model came through Michael Akroyd's presentation entitled "AntiPatterns: Vaccinations against Object Misuse." It was the antithesis of the GoF's design patterns.

The term *anti-pattern* became popular with the authors **Raphael C. Malveau**, **William J. Brown**, **Hays W. "Skip" McCormick**, and **Thomas J. Mowbray** via their famous book titled *Anti Patterns: Refactoring Software, Architectures, and Projects in Crisis* (Robert Ipsen/Wiley, 1998). They said the following:

"Because AntiPatterns have had so many contributors, it would be unfair to assign the original idea for AntiPatterns to a single source. Rather, AntiPatterns are a natural step in complementing the

work of the design pattern movement and extending the design pattern model.”

Examples of Anti-Patterns

These are examples of anti-patterns and the concepts/mindsets behind them:

- **Overuse of Patterns:** Developers use patterns at any cost, regardless of whether they are appropriate or not.
- **God Class:** A big object that tries to control almost everything with many unrelated methods. Inappropriate use of the Mediator pattern may end up with this anti-pattern.
- **Not Invented Here:** I am a big company and I want to build everything from scratch. Although a library is available (developed by a small company), I won't use it. I make everything on my own and once it is developed, I use my brand value to announce “Hey guys, the ultimate library is launched for you.”
- **Zero Means Null:** Consider a case when a developer makes an application form with some mandatory fields. In this form, if you do not enter a number, the developer treats it as a 0. There are some variations of this anti-pattern. For example, some developers like to use -1, 999, or anything like that to represent an inappropriate integer value for a context or to exit from a loop. Sometimes they treat 09/09/9999 as a null date in an application. In these cases, if the user needs to use the numbers -1 or 999 or the date 09/09/9999, they won't be able to do so.
- **Golden Hammer:** Mr. X believes that technology T is the best. When he develops a new system (which requires new learning), he tries to use T even if it is inappropriate. He thinks, “I am old enough and quite busy. I do not need to learn any more technology if I can somehow manage it with T.”
- **Shoot the Messenger:** “I'm already under pressure and the program deadline is approaching. The tester, John, always finds defects, which are hard to fix. Also, John does not like me so he likes to find defects in my code. At this stage, I do not want him to validate my code; he will find more defects and I will miss my target deadline”.
- **Swiss Army Knife:** Demand for a product that will serve every need of a customer, or a drug that will cure all illnesses, or software that

will serve a wide range of customers with varying needs. It does not matter how complex the interface is.

- **Copy-and-Paste Programming:** You need to solve a problem, but you already have a piece of code to deal with a similar situation. So, you take a copy of the old code that is currently working, and you start modifying it if required. But when you start from an existing copy, you essentially inherit all the potential bugs associated with it. Also, if the original code needs to be modified in the future, you may need to apply the modification in multiple places. This approach also violates the Don't Repeat Yourself (DRY) principle.
- **Architects Don't Code:** Mr. X thinks, "I am an architect. My time is valuable. I only show paths or give great lectures on coding. There are enough implementers who should implement my idea." **Architects Play Golf** is a brother of this anti-pattern.
- **Hide and Hover:** Do not expose edit or delete links until a user hovers over the element.
- **If It's Working, Don't Change It:** You identify some poorly structured code that is still working. Some of your colleagues say, "Do not make any change to it because it is still working. Unless it causes big trouble, leave it."
- **Disguised Links and Ads:** A mindset that thinks that to make a profit, it is OK to fool users and earn revenue when they click on a link or an advertisement although they may not get the information.
- **Management by Numbers:** A mindset that thinks that greater numbers of commits, greater numbers of lines of code, or greater numbers of defect fixing, etc. are signs of a great developer.

In this context, I like to mention the following quote by Bill Gates (source:

www.goodreads.com/quotes/536587-measuring-programming-progress-by-lines-of-code-is-like-measuring):

"Measuring programming progress by lines of code is like measuring aircraft building progress by weight."

Bill Gates

Points to Note

- Nowadays you can learn about various anti-patterns from different websites/sources. For example, the following Wikipedia link talks

about various anti-patterns:

<https://en.wikipedia.org/wiki/Anti-pattern>.

- You can also get a detailed list of the anti-pattern catalog at <http://wiki.c2.com/?AntiPatternsCatalog> to learn more.

Types of Anti-Patterns

Anti-patterns can belong to different categories. Even a typical anti-pattern can belong to more than one category. Here are some common classifications:

- **Architectural:** The Swiss Army Knife anti-pattern is an example in this category.
- **Development:** The God Class and the Overuse of Patterns anti-patterns are examples in this category.
- **Management:** The Shoot the Messenger anti-pattern falls in this category.
- **Organizational:** Architects Don't Code and Architects Play Golf belong in this category.
- **User Interface:** Examples include disguised links/ads.

Note Disguised links and advertisements are known as Dark Patterns.

Q&A Session

29.1 How are anti-patterns related to design patterns?

When you use design patterns, you reuse the experiences of others who came before you. When you start blindly using those concepts for the sake of use only, you fall into the trap of *reuse of recurring solutions*. This can lead you to a bad situation in the future where you identify that your return of investment (ROI) keeps decreasing but maintenance costs keep increasing. In simple words, attractive solutions (or patterns) may cause more problems for you in the future.

29.2 A design pattern may turn into an anti-pattern. Is this understanding correct?

Yes, if you apply a design pattern in the wrong context, it can cause more trouble than the problem it solves, and eventually it will turn into an anti-pattern. So, before you start, understanding the nature and context of the problem is very important. If you are not sure, follow the Keep It Simple, Stupid (KISS) principle and make your solution as simple as possible.

29.3 Antipatterns are related to software developers only. Is this understanding correct?

No. The usefulness of an anti-pattern is not limited to developers; it may apply to others also. For example, it is useful to managers and technical architects, too.

29.4 Even if you do not get much benefit from anti-patterns now, they can help you adapt new features easily with fewer maintenance costs in the future. Is this understanding correct?

Yes.

29.5 What are the probable causes of anti-patterns?

They can come from various sources/mindsets. Here are some common examples:

- “We need to deliver the product as soon as possible.”
- “We have a very good relationship with the customer. So, at present, we do not need to analyze the future impact.”
- “I am an expert of reuse. I know design patterns very well.”
- “We use the latest technologies and features to impress our customers. We do not need to care about legacy systems.”
- “More complicated code will reflect my expertise in the subject.”

29.6 Can you mention some symptoms of anti-patterns?

In object-oriented programming, the most common symptom is that your system cannot adopt a new feature easily. Also, you discover that the maintenance costs are continuously increasing. You may also notice that you have lost the power of key object-oriented features like inheritance and polymorphism .

Apart from these, you may notice some (or all) of the following symptoms:

- Use of global variables
- Code duplication
- Limited/no reuse of code
- One big class (God class)
- Presence of a big number of parameterless methods

29.7 What is the remedy if you detect an anti-pattern?

You may need to refactor your code and find a better solution. For example, here are some solutions to avoid the following anti-patterns:

- **Golden Hammer:** You may try to educate Mr. X through proper training.
- **Zero Means Null:** You can use an additional Boolean variable that is more sensible for indicating a null value properly.
- **Management by Numbers:** The counting of numbers is good if you can use them wisely. You cannot judge the ability of a programmer just by the number of defects they fix per week. Quality is also important. A typical example: fixing a simple UI layout is much easier than fixing a critical memory leak in the system. Consider another example. “More tests are passing” does not indicate that your system is more stable unless these tests exercise different code paths/branches.
- **Shoot the Messenger:** Welcome tester John and involve him immediately. Don’t consider him as a rival. You can properly analyze all of his findings and fix the real defects early to avoid last minute surprises.
- **Copy-and-Paste Programming:** Instead of searching for a quick solution, you can refactor your code. You can also make it commonplace to maintain the frequently used methods to avoid duplicates and easier maintenance.
- **Architects Don’t Code:** Involve the architects in some parts of the implementation phase. This can help both the organization and the architects. This activity can give them a clearer picture of the true functionalities of the product. And truly, they should value your effort.

29.8 What do you mean by refactoring?

In the coding world, the term *refactoring* means improving the design of an existing code without changing the external behavior of the system/application. This process helps you get more readable code. At the same time, the code should be more adaptable to new requirements (or change requests) and more maintainable.

Previous chapter

< [28. Criticisms of Design Patterns](#)

Next chapter

[30. FAQ](#) >